

Algorithm for file updates in Python

Project description

Practica de python con jupyter

Open the file that contains the allow list

En este laboratorio técnico, se implementó un script básico en **Python** diseñado para la gestión de permisos en red. El proceso consistió en definir un archivo de origen (**allow_list.txt**) y una lista de control (**remove_list**) que contiene direcciones IP específicas que deben ser restringidas del sistema.

```
In [1]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Display `import_file`
print(import_file)

# Display `remove_list`
print(remove_list)

allow_list.txt
['192.168.97.225', '192.168.158.170', '192.168.201.40', '192.168.58.57']
```

Read the file contents

En esta fase del desarrollo, el script evoluciona para interactuar directamente con el sistema de archivos. La imagen ilustra cómo se automatiza la apertura y lectura de datos externos, un paso crítico para procesar listas de control de acceso de manera dinámica.

```
In [2]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Convert the string into a list

En esta etapa del laboratorio, el enfoque se desplaza hacia la preparación de los datos para un análisis preciso. La imagen muestra un paso fundamental en la ingeniería de datos aplicada a la seguridad: la conversión de contenido plano en estructuras organizadas.

```
In [6]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)
```

Iterate through the remove list

En esta fase del proyecto, el script alcanza un nivel de operatividad funcional mediante la automatización de recorridos de datos. La imagen demuestra el uso de estructuras de control para procesar información de seguridad de forma individual y sistemática.

```
In [6]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)
```

Remove IP addresses that are on the remove list

La imagen muestra la etapa culminante donde el script aplica reglas de decisión para depurar la lista de acceso. En este punto, el programa deja de solo leer datos y comienza a realizar modificaciones basadas en políticas de seguridad predefinidas.

```
In [8]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)
```

Update the file with the revised list of IP addresses

La imagen ilustra el paso final del script: asegurar que los cambios realizados en la memoria se apliquen permanentemente al sistema. En esta etapa, el script actualiza el archivo original con la nueva lista depurada de direcciones IP autorizadas.

```

In [9]: # Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)

```

Summary

Como conclusión, este laboratorio técnico demuestra la eficacia de Python como herramienta esencial para la automatización de la ciberseguridad y la gestión de redes. A través del desarrollo del script, se ha logrado integrar un ciclo completo de administración de datos que comienza con la ingesta de archivos planos y culmina con la actualización persistente de las políticas de acceso en el sistema. La implementación de técnicas de ingeniería de datos, como la tokenización mediante el método `.split()` y el uso de estructuras iterativas y condicionales, permite procesar grandes volúmenes de direcciones IP con una precisión que elimina el margen de error humano inherente a las tareas manuales.

Finalmente, el uso de sentencias de manejo de recursos como `with open()` y la reconstrucción de información mediante `.join()` aseguran que el flujo de trabajo sea no solo funcional, sino también eficiente y seguro desde el punto de vista del rendimiento del sistema. Este proceso de depuración automática de listas de acceso garantiza que las restricciones de red se mantengan actualizadas en tiempo real, fortaleciendo la postura de seguridad de cualquier

organización al permitir una respuesta ágil y escalable ante la necesidad de revocar permisos a nodos comprometidos.